

# CPSC 405, Operating Systems and Programming

Lecture 18 – Mid-term Review

# Mid-term

---

- ▶ Mid-term is worth 500 points
- ▶ Exam is 25 February during class – 10am-11:50am
  - ▶ If you have an a documented need for more time, and you want more than two hours you must see me so I can find accommodations
- ▶ 50 questions
  - ▶ Mostly multiple choice
  - ▶ Code samples – what does this output
- ▶ Mid-term is taken from the lecture notes (1-17) and Chapters 1-4 of the text

# Topics through the Mid-term

---

- ▶ **Modifying xv6**
- ▶ **Memory Abstractions/OS Design**
- ▶ **System Programming in C/xv6**
  - ▶ System Calls and Programming
  - ▶ Calling Conventions
- ▶ **Traps - System Calls and Exceptions**
- ▶ **Virtual Memory and Page Tables**

# Modifying xv6

---

- ▶ Steps to add a user program in xv6
  - ▶ add the .c program to /user directory
  - ▶ Update the \$UPROGS variable in the makefile
  - ▶ Make qemu from the xv6\_labs directory
- ▶ Steps to add a system call in xv6
  - A. Set up the system call test in user space
  - B. System call stub setup
    1. Add the system call to the user API
    2. Add a system call number to kernel/syscall.h
    3. Add call information for the system call for trap
    4. Add a stub for ecall in user/usys.pl
    5. Implement the system call
    6. Add call information to syscall.c
  - C. Test the sysinfo system call

# Memory's many layers of abstraction

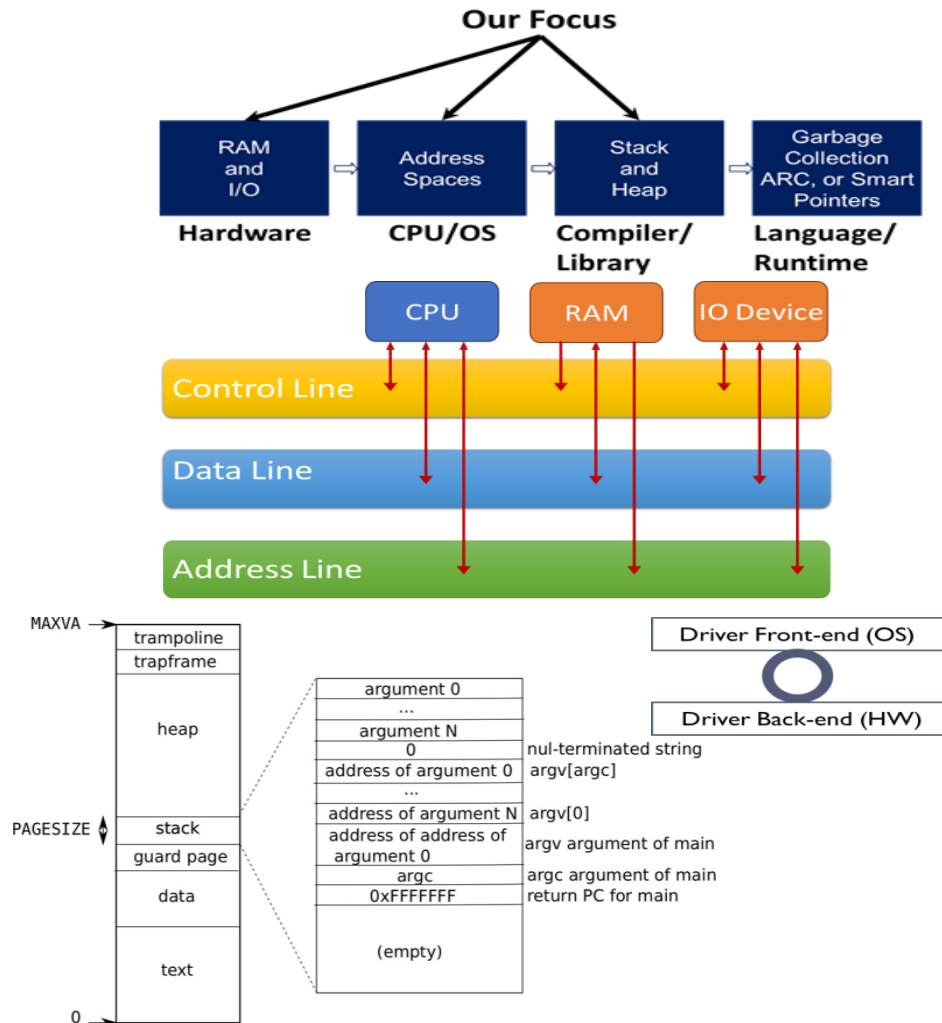


Figure 3.4: Memory layout of a user process with its initial stack.

## ▶ Hardware layer

- ▶ A bus transfers data between components in the computer
- ▶ Bus interface is too low-level to do anything useful

## ▶ CPU/OS layer

- ▶ The address is the array index in bytes
- ▶ Programmer can then interact with IO devices through loads and stores

## ▶ Compiler/Library Layer

- ▶ An address space is also too low-level
- ▶ How can we decide where in the array to store things?

# OS Attributes

---

## ▶ OS Attributes

- ▶ Kernel – monolithic, micro, none
- ▶ Control/Manage Hardware
- ▶ Hardware Abstractions
- ▶ Provide interfaces (an API) that use the abstractions so programmers can create programs that execute as processes

## ▶ CPUs provide hardware support for strong isolation

- ▶ Machine mode: a mode for booting and configuring machine.
- ▶ Supervisor mode: a mode for executing a monolithic kernel.
- ▶ User mode: mode for user-written programs and process execution
- ▶ OS use kernel and user modes

## ▶ Moving between modes

- ▶ `ecall`, `sret`, `mret`

# Process Structure

- ▶ The process is the main unit of isolation
  - ▶ Processes represent a thread of execution
  - ▶ Have their own virtual memory mapping
    - ▶ Appears as a private address space
  - ▶ Dedicated user text (code) and data – statics and globals
  - ▶ Dedicated user stack (grows down)
  - ▶ Dedicated heap (grows up)
  - ▶ A dedicated “trapframe”
    - ▶ writeable area for transition between user & kernel
  - ▶ A shared “trampoline”
    - ▶ code used to transition between user & kernel
- ▶ Process abstraction - appears to run on its own private machine
  - ▶ Kernel time-slicing of threads
  - ▶ Kernel uses page tables to implement virtual address space
- ▶ In the kernel, a process is represented by a struct proc
  - ▶ This structure maintains resources dedicated to process: page table, state, stack, etc.

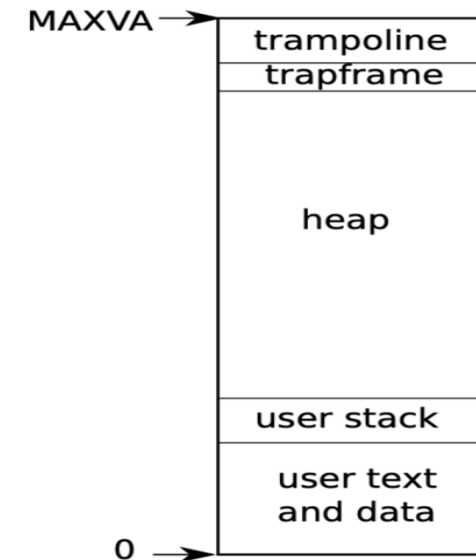


Figure 2.3: Layout of a process's virtual address space

## Review Questions 1 - Memory Abstractions/OS Design

---

1. What is the qemu emulator emulating for xv6?
2. What are the three memory abstractions we focus on in this course?
3. What is an address space and why do we need one?
4. Why do we have permission in our address spaces and what happens if the CPU loads or stores an address without permission?
5. When is it better to use a stack versus the heap?
6. xv6 is ~6KSLOC, why is it considered a monolithic kernel?
7. How does an OS isolate processes?
8. What is the main security requirement in kernel design?
9. The software running in supervisor mode is called?



# Bitwise operations

- ▶ `0b10001 & 0b10000 == 0b10000` // bitwise AND
- ▶ `0b10001 | 0b10000 == 0b10001` // bitwise inclusive OR
- ▶ `0b10001 ^ 0b10000 == 0b00001` // bitwise exclusive OR
- ▶ `~0b1000 == 0b011` // one's compliment (unary)
- ▶ `#define PTE2PA(pte) (((pte) >> 10) << 12)`
  - ▶ `VA = 0x21fd8c17222`
  - ▶ `Offset = 222 (0b0010 0010 0010)`
  - ▶ `Page table entry is 0x21fd8c17 (0b10 0001 1111 1101 0000 0000 0000 0000)`
  - ▶ `pte >> 10 : 0b10 0001 1111 1101 0000 00`
  - ▶ `0b1000 0111 1111 0100 0000`
  - ▶ `8 7 f 8 0`
  - ▶ `pte << 12 : 0b1000 0111 1111 0110 0011 0000 0000 0000`
  - ▶ `8 7 f 6 3 0 0 0`
  - ▶ `pa is 0x87f63000 add the offset of 222`
  - ▶ `PA = 0x87f63222`

|   |   | AND | OR | XOR | ~ |
|---|---|-----|----|-----|---|
| 0 | 0 | 0   | 0  | 0   | 1 |
| 0 | 1 | 0   | 1  | 1   | 1 |
| 1 | 0 | 0   | 1  | 1   | 0 |
| 1 | 1 | 1   | 1  | 0   | 0 |

# Integer Bit Mask

---

## ► In C,

- `1 << SYS_fork` is equivalent to `1 << 1`
  - which performs a left shift operation on the binary representation of the number 1 (0001)
- Left shift operator (`<<`) shifts the bits of the left operand to the left by the number of positions specified by the right operand
  - Shifting 0001 one position to the left results in 0010 - binary representation of 2
  - `1 << SYS_fork` evaluates to 2 or 0010
  - `1 << SYS_close == 1 << 21` evaluates to 2,097,152

## ► Thus,

```
10 (21 == 2 )- fork
100 (22 == 4)- exit
1000 (23 == 8)- wait
10000 (24 == 16)- pipe
100000 (25 == 32) - read
1000000 (26 == 64) - kill
...
10000000000000000000000000 (221 == 2,097,152) - close
```

# Review Questions2 - System Programming

---

1. What does `open ()` return?
2. What is the first argument to `read()` and `write()`? Return?
3. What are the three arguments passed to the `read()` and `write()` system calls?
4. What is a file descriptor?
5. What is file descriptor 0? 1? 2? 3+?
6. What does the `fork()` function call do?
7. What does the `exec()` function call do?
8. Are file descriptors preserved after `exec()`?
9. How do you redirect `stdout`?
10. What is a pipe?
11. What is the pipe system call?
12. Relate the read/write end of a pipe to `stdin/stdout`?
13. As UNIX uses a file abstraction for devices, why is a pipe better than using files for two processes to communicate?

# First Boot

---

## Boot code:

- ▶ RISC-V powers on
- ▶ RISC-V runs a boot loader which is stored in read-only memory
- ▶ Bootloader loads xv6 kernel into memory
  - ▶ Kernel is loaded at 0x8000000
  - ▶ 0x0:0x8000000 contains I/O devices
- ▶ Transfers control to the **kernel**
- ▶ CPU executes xv6 starting at the kernel's `_entry` routine
  - ▶ Sets up a stack so xv6 can run C code
  - ▶ Sets stack pointer register to `stack0+4096`, stack grows down
  - ▶ `_entry` calls into C code `start()`
- ▶ `start()` performs configurations
- ▶ `start()` set return address to `main()` by writing `main`'s address into register `mepc`
- ▶ `start()` disables paging, initialize clock interrupts, calls `mret`
  - ▶ Changes the mode to supervisor
  - ▶ Program counter changes to `main()`

## Kernel's main:

- ▶ `main()` does all initialization of devices sets up initial page tables and initializes timer interrupts
- ▶ At this point `main ()` creates first process by calling `userinit()`
  - ▶ This function loads an assembly file 'initcode.S' and execute instructions
  - ▶ It invokes the `exec` system call re-entering kernel mode
  - ▶ It execs `/init`
- ▶ `/init`
  - ▶ This program has pid 1, opens file descriptor 0, 1, 2
  - ▶ Forks a child
  - ▶ Child execs a console shell.
  - ▶ Parent loops and handles orphaned zombies repeatedly
- ▶ The system is up

# Review Questions 3 - System Calls and Programming

---

1. When are file descriptors 0, 1, and 2 opened?
2. What program opens file descriptors 0, 1, and 2?
3. How does a user process open file descriptors 0, 1, and 2?
4. What program has pid of 1?
5. What does init do when it finishes initializing the system?
6. What is the difference in user and kernel space?
7. What happens if an application in user mode attempts to execute a privileged instruction?

# System call arguments

---

- ▶ User code calls system call wrapper functions
- ▶ The arguments are initially in registers
  - ▶ following the RISC-V C calling conventions
- ▶ The kernel trap code saves user registers to the current process's trapframe, where kernel code can find them
- ▶ The kernel functions `argint`, `argaddr`, and `argfd` retrieve the n'th system call argument from the trapframe as an integer, pointer, or a file descriptor
  - ▶ call `argraw` to retrieve the appropriate saved user register

# Review Questions 4 - System Calls, Programming and Calling Conventions

---

1. What is trampoline in xv6?
2. What is trapframe in xv6?
3. Can an application invoke a kernel function directly? Why or why not.
4. What is the RISC-V base ISA?
5. Why follow calling conventions?
6. If you are following calling conventions, where does the caller put arguments?
7. If you are following calling conventions, where does the callee store the return value?
8. What are the three events that cause a suspension in the current thread of execution and what are these three events called in xv6?
9. If we follow calling conventions, how is the kernel passed arguments?

# The Trap Handling Algorithm - entry

---

## ▶ Userspace

- ▶ `uservec` - saves the user's registers and switches to the supervisor page table
  - ▶ saves the user's registers to `TRAPFRAME`
  - ▶ `TRAPFRAME` contains current process's status (kernel stack, CPU hart id) in addition to address of `usertrap` function, and the kernel page table
  - ▶ switches `satp` to the kernel page table
  - ▶ Calls `usertrap`
- ▶ `usertrap` - determines the cause of the trap, process it, and return
  - ▶ Set up kernel trap vector – `stvec` (trap handled by `kernelvec`)
  - ▶ Saves `sepc` (the saved user program counter)
  - ▶ Determine the cause of the trap and routes it appropriately
    - call `syscall` for system calls ; call `devintr` for device interrupts; kill the process for other exceptions
  - ▶ Trap is processed – call `usertrapret` – sets `stvec` to `uservec`

## ▶ Kernel space

- ▶ trap vector is `stvec`, page table does not have to be changed
- ▶ context is stored on the kernel stack - `kernelvec`
- ▶ There is a similar nesting-dolls set of routines for calling and returning from the trap handler
  - ▶ `kernelvec` saves the registers, jumps to `kerneltrap`
  - ▶ `kerneltrap` calls `devintr` to handle device interrupts; calls `panic` to handle exceptions



# Review Questions 5 - Traps

---

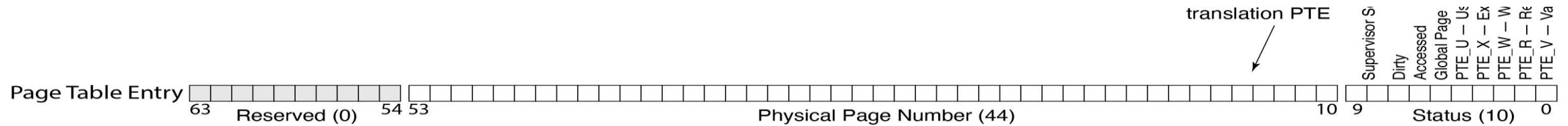
1. What is the RISC-V instruction that controls access to Supervisor mode?
2. What are events that cause a suspension in the current thread of execution called?
3. The RISC-V hardware doesn't switch page tables during a trap, how does xv6 avoid a kernel crash accessing the user's page table?
4. What CSR holds the physical address of the code that will handle traps?
5. What is the user mode interrupt handler routine?
6. What is the supervisor mode interrupt handler routine?
7. Where is the code that stvec points to?
8. How does xv6 swap the user and kernel page tables and execute uservec code for all processes?
9. What process artifact supports swapping of contexts between user process and kernel?
10. What are the trap handlers in userspace? kernelspace?

# Today's Problem

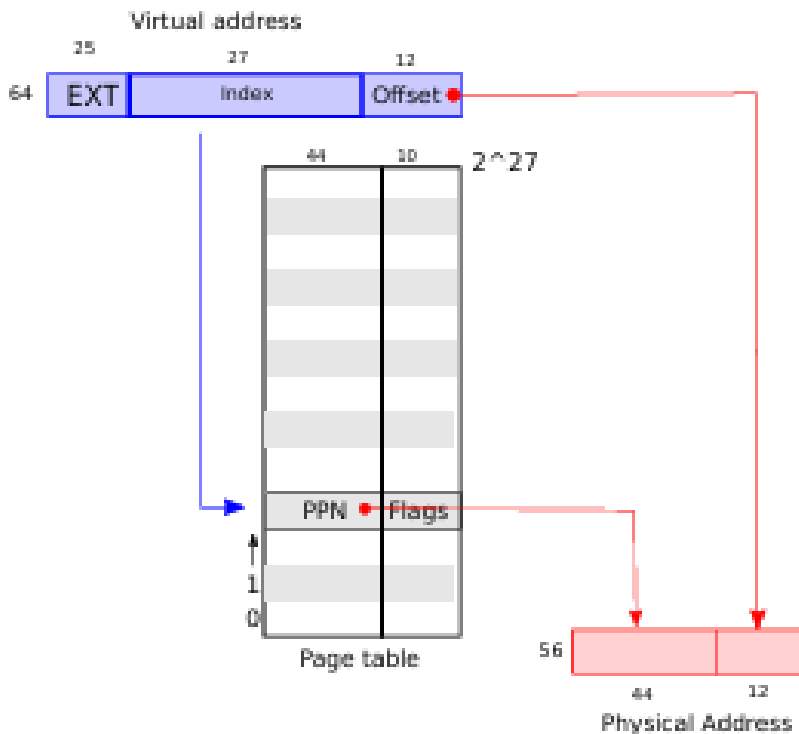
---

- ▶ Suppose the shell has a bug
  - ▶ Sometimes it writes to a random memory address
    - ▶ Memory is just an array of bytes - apps and kernel in same memory
    - ▶ how can we keep it from wrecking the kernel?
    - ▶ from wrecking other processes?
  - ▶ We want isolated address spaces
    - ▶ Each process has its own memory
      - it can read and write its own memory
      - it cannot read or write anything else
- ▶ Virtual memory multiplexes several address spaces onto one physical memory while maintaining isolation

# RISC-V sv39 Page Table Entry



- ▶ Virtual page numbers are 27-bits long
- ▶ Translation for the virtual number bits get a PTE
- ▶ Page Table Entry: 64-bits (8-bytes)
  - ▶ A 44-bit physical page number
  - ▶ Metadata: 10 status bits describing page entry (valid) or the page itself (read, write, execute, user, dirty, et al.)



# RISC-V sv39 Page Table

- ▶ The page table is implemented as a 3-level tree
  - ▶ The 27-bit virtual address is split into three 9-bit chunks
- ▶ Rooted at a single 4KB page that contains physical addresses for similar pages in the second level of the tree - 1 page table
  - ▶ The CPU expects to find the page table root at the memory address in the satp CSR
- ▶ Each page contains 512 page table entries - 512 page tables
- ▶ Each second level page does the same for a third level
- ▶ Each of which addresses a 512-entry page table - 262,144 page tables
- ▶ Each leaf page contains the actual mapping to the physical address and flags
- ▶ The page table is stored in physical memory
  - ▶ An optimization to store a 1 GB lookup table in 4096-byte chunks
  - ▶ 262,657 4KB pages total
  - ▶ 513 additional pages are required

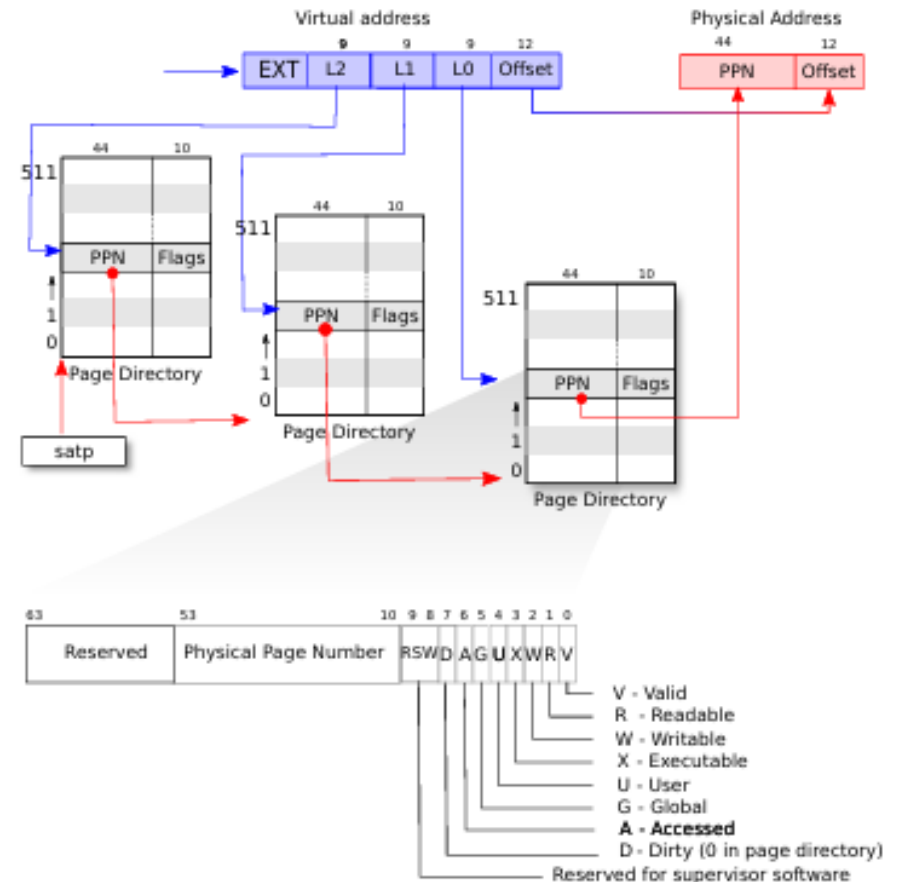


Figure 3.2: RISC-V address translation details.

# A Symbiotic Relationship

---

- ▶ Only the operating system can make a tree:
  - ▶ The page table is constructed by the OS
    - ▶ As the machine boots, the kernel's page table is built in `kvminit` (`kernel/vm.c`)
    - ▶ As processes are constructed, page tables are built in `proc_pagetable` (`kernel/proc.c`)
  - ▶ Page tables are maintained by the OS
    - ▶ Memory allocation leads to new virtual memory addresses for allocated physical page
    - ▶ Sets PTE status bits: e.g. when loading a new executable
- ▶ The hardware uses the tree to form translations
  - ▶ The `satp` CSR register holds the physical address of the root block of the tree
  - ▶ Hardware
    - ▶ Assumes the tree is in memory and will walk the tree to form the translations
      - Ideally: everything's in memory, and cached in `LI$`; done in 3 reads
      - If any of the three PTEs required to translate an address is not present, the paging hardware raises a page-fault exception, leaving it up to the kernel to handle the exception

# Summary - xv6 kernel memory mapping

---

- ▶ The kernel manages all of physical memory
- ▶ The kernel maintains one page table per process and one page table for itself
- ▶ The kernel uses direct mapping
  - ▶ Identical physical and virtual memory address
- ▶ The trampoline page and kernel stack pages are kernel virtual addresses that aren't direct-mapped
  - ▶ Top page of virtual memory is call the trampoline
    - ▶ A directly mapped shared block of mode-switching code
    - ▶ Several processes may be executing code in the kernel, from different cores
  - ▶ Kernel shares one block (the trap frame) with each process
    - ▶ Writeable by trampoline
  - ▶ Kernel has one block of stack for each process
- ▶ Kernel places guard pages between critical areas
- ▶ Kernel sets up its own address space
  - ▶ paging is not enabled when kernel starts, so addresses are physical
  - ▶ kernel must first create its own page table - `kvmmake()` in `vm.c`
  - ▶ `kvmmap()` adds PTEs to a page table under construction

# User Process stacks

- ▶ Kernel switches page tables (i.e., sets satp) when switching processes
- ▶ xv6 user process stacks are limited to being a single page in size
  - ▶ bounded with a guard page
- ▶ **Stack Guard Page**
  - ▶ Stack has a finite size
  - ▶ Push too much data and it could overflow into adjacent memory
  - ▶ Install an empty mapping (PTE\_V cleared) at the bottom of the stack
  - ▶ There is no memory allocated for a guard page – just a virt. mem. mapping

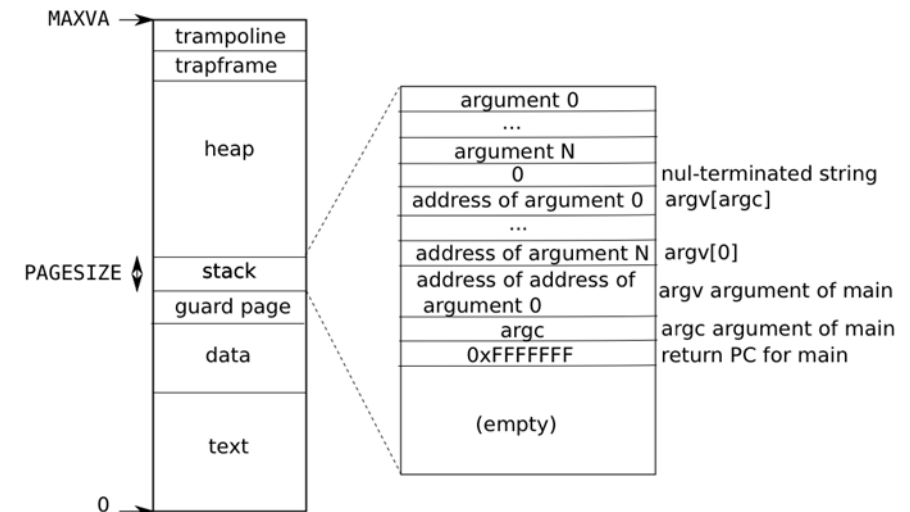


Figure 3.4: A process's user address space, with its initial stack.

# Page Table Review

---

- ▶ The primary problem with segmented addressing is that it can lead to significant external fragmentation in memory
  - ▶ external fragmentation in memory, where free memory space is divided into small, unusable chunks
  - ▶ due to the variable sizes of segments, resulting in wasted memory and decreased performance
- ▶ Paging systems address external fragmentation by dividing memory into smaller, fixed-size units,
  - ▶ generally provide better memory utilization and reduces external fragmentation by allowing for more granular memory allocation
- ▶ Paging solutions, however, suffer from the requirement of contiguous memory which is solved with a multilevel page table
- ▶ Multilevel page tables effectively reduces the overall size of the page table needed for large virtual address spaces
  - ▶ it can introduce additional complexity and overhead during memory access to find the correct physical address, potentially slowing down memory operations
- ▶ The solution to this problem is a TLB in the MMU, which suffers from
  - ▶ There is only one TLB for all page tables
  - ▶ Limited physical size in hardware
- ▶ TLB can be made more effective by implementing superpages.



# Review Questions 6

---

1. What is the purpose of virtual memory?
2. Processor reference memory through a system of \_\_\_\_\_ addresses.
3. RAM accesses memory through a system of \_\_\_\_\_ addresses.
4. Is virtual memory a physical object?
5. The kernel uses direct mapping except for which pages?
6. The RISV-sv39 processor supports \_\_page-table levels.
7. There are \_\_\_\_\_ bytes in the riscv-39 page frame.
8. There are \_\_\_\_\_ bytes are in a riscv-39 page table entry.
9. There are \_\_\_\_\_ page table entries are in a page frame.
10. There are \_\_\_\_\_ bits in the riscv-sv39 virtual address.
11. How does paging fix segmented address' significant external memory fragmentation?
12. What is the primary problem with Paging systems and how do we solve it?
13. What is the problem with a multilevel page table and how do we solve it?